TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A LAB REPORT
ON

Machine Learning Pipeline: Generative vs. Discriminative Models, Model Capacity, and Cross-Entropy on the Iris Dataset

SUBMITTED BY:

Name: Aditya Kumar Shah

Roll No.: 080BCT011

Lab No.: 08

SUBMITTED TO:

Department of Electronics
and Computer Engineering

1. Objectives

1. To build a complete machine learning pipeline on the Iris dataset, splitting the data into training, validation, and test sets, and to understand why the test set must be kept separate from model/hyperparameter selection.

2. To train and compare a generative model (Gaussian Naive Bayes) and a discriminative model (Logistic Regression) on the same data and evaluate them fairly on a validation set.

3. To study model capacity using Decision Tree classifiers of varying `max_depth`, and to identify settings that underfit and settings that overfit.

4. To select a hyperparameter using validation performance only, and to report an honest, single evaluation of the chosen model on the held-out test set.

5. To relate the observed training/validation accuracies to the concepts of bias and variance.

6. To compute cross-entropy (log loss) for a probabilistic classifier and to examine how confidently correct and confidently wrong predictions contribute to it.

7. To reflect on the practical difference between Maximum Likelihood Estimation (MLE) and Maximum A Posteriori (MAP) estimation, and to illustrate it through the effect of regularization on Logistic Regression.

2. Theoretical Background

A machine learning pipeline consists of preparing data, choosing a model, training it, validating it, and finally evaluating it on unseen data. The available data is typically split into a **training set** (used to fit model parameters), a **validation set** (used to compare models and choose hyperparameters), and a **test set** (kept aside and used only once, for a final, unbiased estimate of generalization performance).

A **generative** model, such as Gaussian Naive Bayes, learns the class-conditional distribution of the data, $p(x \mid y)$, and uses Bayes' rule to obtain $p(y \mid x)$. A **discriminative** model, such as Logistic Regression, models $p(y \mid x)$ directly without describing how the input data itself is generated.

Model capacity describes how flexible a model is. A model with too little capacity cannot represent the true decision boundary and **underfits** – it performs poorly on both training and validation data. A model with too much capacity can memorize the training data, including its noise, and **overfits** – it achieves very high training accuracy but a noticeably lower validation accuracy. **Bias** refers to systematic error caused by an overly simple model (associated with underfitting), while **variance** refers to sensitivity to the particular training sample (associated with overfitting).

Cross-entropy (log loss) for a binary outcome is

$$\text{Cost}(h(x), y) = -y \log(h(x)) - (1 - y) \log(1 - h(x)),$$

which generalizes to the multi-class negative log-likelihood $-\log \hat{p}_y$, where $\hat{p}_y$ is the probability the model assigns to the true class y. Because of the logarithm, a prediction that

is confidently correct ($\hat{p}_y \to 1$) contributes a loss near 0, whereas a prediction that is confidently wrong ($\hat{p}_y \to 0$) contributes a loss that diverges towards infinity.

Maximum Likelihood Estimation (MLE) chooses the parameters θ that maximize the probability of the observed data, $\theta^{\text{MLE}} = \arg\max_\theta \prod_i p(x^{(i)} \mid \theta)$. **Maximum A Posteriori (MAP)** estimation additionally multiplies in a prior $p(\theta)$, $\theta^{\text{MAP}} = \arg\max_\theta \prod_i p(x^{(i)} \mid \theta)\, p(\theta)$, which in Logistic Regression corresponds to adding an L_2 regularization penalty that pulls the weights towards zero.

3. Assignment 1 — Building the Pipeline

The Iris dataset (150 samples, 4 numerical features, 3 classes) was loaded from `scikit-learn` and split into training, validation, and test sets using stratified sampling, exactly as specified in the lab sheet: first 80%/20% into a temporary set and a held-out test set, then the temporary set split 75%/25% into training and validation sets. All features were standardized using a `StandardScaler` fitted only on the training set.

Split	No. of samples	Percentage of total
Training	90	60%
Validation	30	20%
Test	30	20%
Total	150	100%

The test set must not be used while choosing a model or tuning hyperparameters because doing so would let information about the final evaluation data leak into the model-selection process; the reported performance would then be an overly optimistic estimate of how the model behaves on genuinely unseen data. Keeping the test set completely untouched until the very end, and using the validation set for all comparisons, keeps the final test accuracy an honest measure of generalization.

4. Assignment 2 — Generative vs. Discriminative Model

Gaussian Naive Bayes (generative) and Logistic Regression (discriminative) were trained on the standardized training set and evaluated on the validation set.

Model	Validation accuracy
Gaussian Naive Bayes (generative)	0.933 (28/30)
Logistic Regression (discriminative)	0.933 (28/30)

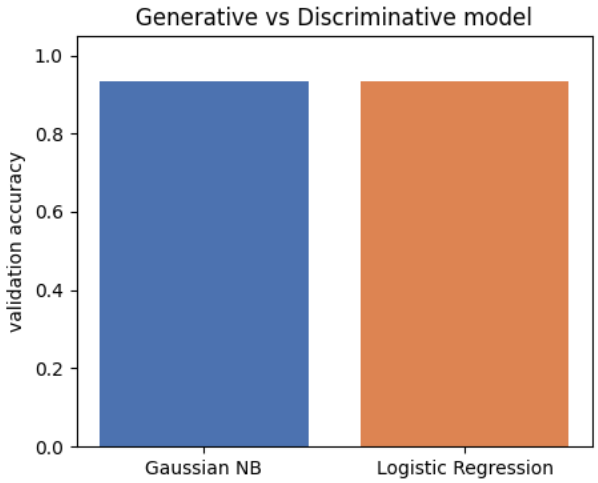

Figure 1: Validation accuracy of Gaussian Naive Bayes (generative) versus Logistic Regression (discriminative) on the Iris dataset.

On this particular split, the two models tie at 93.3% validation accuracy. This is not surprising for the Iris dataset: its three classes are close to linearly separable and roughly Gaussian within each class, which is exactly the setting in which a simple generative model (Gaussian Naive Bayes) and a linear discriminative model (Logistic Regression) are expected to behave similarly. Naive Bayes has the additional advantage of modelling each class's distribution explicitly, but its independence assumption between features is not strictly true for Iris (e.g. petal length and petal width are correlated), which is why it does not outperform Logistic Regression here despite that extra structure.

5. Assignment 3 — Observing Model Capacity

Decision Tree classifiers were trained with `max_depth` $\in \{1, 3, 5, \text{None}\}$, and the training and validation accuracy of each were recorded.

max_depth	**Training accuracy**	**Validation accuracy**
1	0.667	0.667
3	0.989	0.900
5	1.000	0.933
None	1.000	0.933

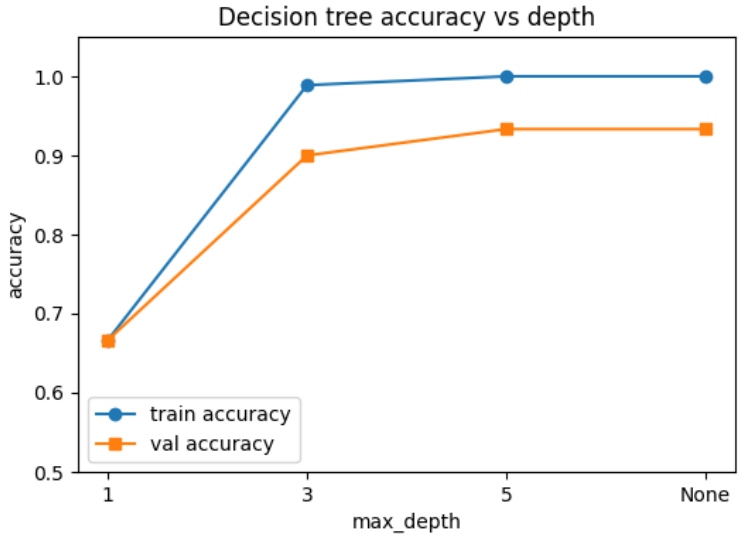

Figure 2: Training vs. validation accuracy of Decision Trees of increasing capacity (max_depth).

Underfitting: `max_depth`=1 clearly underfits. A single split can only separate one class from the other two, so both the training accuracy (0.667) and the validation accuracy (0.667) are low and nearly equal – the model is too simple to capture the structure of the data even on the data it was trained on.

Overfitting: `max_depth`=None (a fully grown, unrestricted tree) shows the classic overfitting signature: it reaches 100% training accuracy by memorizing every training example, while its validation accuracy (0.933) is measurably lower. Because Iris is a small, well-separated dataset, the train–validation gap is modest here, but the tree still uses far more splits/leaves than are needed – `max_depth`=5 reaches the identical validation accuracy (0.933) with a much smaller, more constrained tree, confirming that the extra capacity of the unrestricted tree is not buying any real generalization benefit.

6. Assignment 4 — Selecting a Hyperparameter with Validation Data

Using the validation accuracies from Assignment 3, `max_depth`=5 and `max_depth`=None are tied for the best validation accuracy (0.933). Following the principle of preferring the simplest model that performs best (an application of Occam's razor), `max_depth`=5 was selected as the final hyperparameter. This model was then evaluated *exactly once* on the previously untouched test set.

Quantity	**Value**
Selected `max_depth`	5
Validation accuracy at selection	0.933
Final test accuracy	**0.933**

The final test accuracy (0.933) is close to the validation accuracy that guided the selection, which is what is expected when the validation set is representative of the test distribution and has not been over-used for tuning.

7. Assignment 5 — Bias and Variance Discussion

Model	Training accuracy	Validation accuracy
Shallow tree (max_depth=1)	0.667	0.667
Deep tree (max_depth=None)	1.000	0.933

The shallow tree (`max_depth`=1) is the higher-**bias** model: its training accuracy is low, meaning it makes systematic errors even on the data it has already seen, because a single decision boundary is too simple to represent a 3-class problem. It cannot be fixed by seeing more data of the same kind – its error comes from the model's own limited assumptions, not from noise.

The deep tree (`max_depth`=None) is the higher-**variance** model: it fits the training set perfectly (accuracy 1.0) but its validation accuracy is lower, showing that some of what it "learned" is specific to the particular training sample rather than the true underlying pattern – a different training sample would likely produce a noticeably different tree. This train–validation gap (1.0 vs. 0.933), even though modest on the easy Iris dataset, is the signature of variance.

8. Assignment 6 — Cross-Entropy

Class probabilities from the trained Logistic Regression model were obtained with `predict_proba()` on the validation set, and the validation log loss was computed with scikit-learn's `log_loss` function.

Quantity	Value
Validation log loss (mean cross-entropy)	0.215

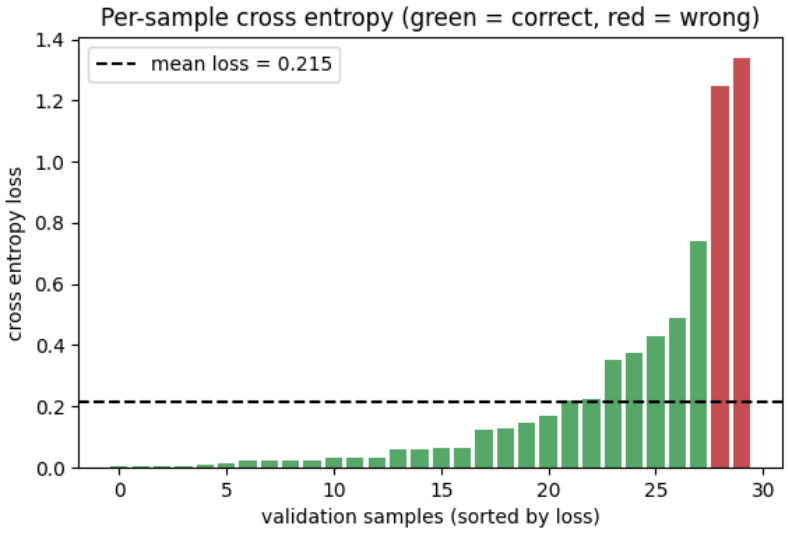

Figure 3: Per-sample cross-entropy loss on the validation set, sorted from lowest to highest. Correctly classified samples are shown in green, misclassified samples in red.

Sample	True class	Pred. prob. of true class	Sample loss
Confidently correct (#12)	setosa	0.997	0.003
Confidently wrong (#10)	versicolor	0.262 (pred. virginica)	1.340

The confidently correct prediction assigns 99.7% probability to the true class (setosa), so its cross-entropy contribution is close to zero ($-\log(0.997) \approx 0.003$). The misclassified sample, in contrast, assigns only 26.2% probability to the true class (versicolor) – the model instead favoured virginica – producing a contribution of $-\log(0.262) \approx 1.340$, roughly 390 times larger than the confident correct prediction's loss. This asymmetry is exactly what cross-entropy is designed to do: it barely penalizes confident, correct predictions, but heavily penalizes predictions that are both wrong and overconfident.

9. Assignment 7 — MLE and MAP Reflection

To illustrate the difference between MLE and MAP, Logistic Regression was retrained with several values of the inverse regularization strength C (a small C corresponds to a strong prior pulling weights towards zero, i.e. a more MAP-like fit; a large C approaches an unregularized MLE fit).

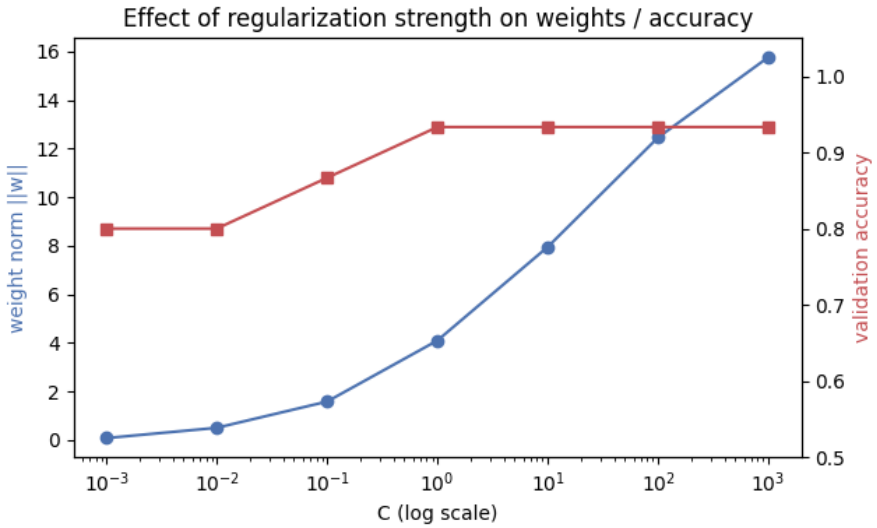

Figure 4: Effect of the regularization strength on the learned weight magnitude and on validation accuracy. Small C (strong prior, MAP-like) shrinks the weights and can underfit; large C (weak prior, MLE-like) lets the weights grow and fits the training distribution more closely.

As C decreases (stronger regularization, more MAP-like), the norm of the learned weight vector shrinks steadily, from about 15.8 at $C = 1000$ down to about 0.08 at $C = 0.001$; validation accuracy also drops in the heavily regularized regime (to 0.80 at $C \leq 0.01$) because the prior is now strong enough to pull the decision boundary away from the data. MLE finds the parameters that make the observed training data as likely as possible, using only the likelihood term $p(D \mid \theta)$, so with enough data and no constraints it can overfit if the model has high capacity. MAP additionally multiplies in a prior $p(\theta)$ that encodes a preference – here, for smaller weights – so it trades a small amount of fit

to the training data for a simpler, more regularized model. In practice, MLE is the "let the data speak for itself" estimate, while MAP is what you get when you also tell the model what kind of parameter values you expect to see *before* looking at the data; the strength of that prior is itself a hyperparameter (here, $1/C$) that must be tuned, typically using a validation set, so it is neither too weak (overfitting, MLE-like) nor too strong (underfitting).

10. Questions to be Answered

1. **Why can a model have very high training accuracy but lower validation accuracy?** This happens when the model has enough capacity to memorize patterns – including noise and idiosyncrasies – that are specific to the training set. Those memorized patterns do not generalize, so performance drops on the validation set, which the model has never seen. This is overfitting, and the size of the train–validation gap is a direct measure of it.

2. **Why is a validation set useful when choosing hyperparameters?** It gives an estimate of generalization performance for each candidate hyperparameter setting without ever touching the test set. If hyperparameters were tuned directly against the test set instead, the test accuracy would no longer be an unbiased estimate of performance on new data, since the choice would already be fitted to that specific set.

3. **How are underfitting, high bias, and low model capacity related?** A model with low capacity (e.g. a decision stump) cannot represent the true decision boundary of the problem, no matter how much training data it sees. This systematic, unavoidable error is called bias, and it shows up as underfitting: both training and validation accuracy stay low and close to each other, as seen with `max_depth`=1 in Assignment 3.

4. **How are overfitting, high variance, and high model capacity related?** A model with high capacity (e.g. an unrestricted decision tree) can fit the training data extremely closely, including its noise. Because that fit depends heavily on the particular sample of training data used, a different training sample would produce a noticeably different model – this sensitivity is variance. It shows up as overfitting: high training accuracy but a noticeably lower validation accuracy, as seen with `max_depth`=None in Assignment 3.

5. **What extra information does MAP use that MLE does not?** MAP uses a prior distribution $p(\theta)$ over the model parameters, representing beliefs about which parameter values are more plausible before seeing the data (e.g. that smaller weights are more likely). MLE uses only the likelihood of the observed data, $p(D \mid \theta)$, which is equivalent to MAP with a flat (uninformative) prior.

6. **Why does cross-entropy heavily penalize a confident but wrong prediction?** Cross-entropy for a single prediction is $-\log(\hat{p}_y)$, where $\hat{p}_y$ is the probability assigned to the true class. As a confident-but-wrong prediction assigns $\hat{p}_y$ close to 0 to the true class, $-\log(\hat{p}_y)$ grows without bound, producing a very large loss – exactly what was observed for the misclassified sample in Assignment 6, whose loss (1.340) was nearly 400 times larger than that of a confident, correct prediction.

11. Conclusion

A complete machine learning pipeline was built on the Iris dataset, from stratified training/validation/test splitting through model comparison, capacity analysis, hyperparameter selection, and loss analysis. Gaussian Naive Bayes (generative) and Logistic Regression (discriminative) performed identically on the validation set, consistent with Iris being an easy, near-linearly-separable problem. Decision Tree capacity experiments showed the expected underfitting behaviour at `max_depth`=1 and the expected overfitting signature (100% training accuracy with a lower validation accuracy) at `max_depth`=None, motivating the selection of `max_depth`=5 via the validation set and a final, single evaluation of 0.933 test accuracy. The bias–variance framing mapped directly onto these two extremes: the shallow tree exhibited high bias, and the deep tree exhibited high variance. Cross-entropy analysis of the Logistic Regression model confirmed that confidently correct predictions contribute almost no loss while confidently wrong predictions are penalized heavily, and the MLE/MAP illustration showed how regularization strength trades off between fitting the training data closely (MLE-like) and constraining the parameters toward a simpler prior belief (MAP-like).

The full Python source used to produce all the results and figures in this report is listed in the appendix below, and is also included as `code/pipeline.py` along with the generated plots and `results.json` in the `outputs` folder.

Appendix: Python Source Code

```
# AI Lab 8 - Iris dataset ML pipeline (Assignments 1-7)

import json
import numpy as np
import matplotlib
matplotlib.use("Agg")  # so plots save fine when run from terminal, no popup window
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, log_loss

# ---------- Assignment 1: load data and build train/val/test split ----------

iris = load_iris()
X = iris.data
y = iris.target
class_names = iris.target_names

# first pull out 20% as the final test set
```

```
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
# then split what's left into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=
       y_temp
)

print("train samples:", len(X_train))
print("val samples:", len(X_val))
print("test samples:", len(X_test))

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# ---------- Assignment 2: Gaussian Naive Bayes vs Logistic
   Regression ----------

gnb = GaussianNB()
gnb.fit(X_train, y_train)
gnb_acc = accuracy_score(y_val, gnb.predict(X_val))
print("GaussianNB val accuracy:", gnb_acc)

logreg = LogisticRegression(max_iter=1000, random_state=42)
logreg.fit(X_train, y_train)
logreg_acc = accuracy_score(y_val, logreg.predict(X_val))
print("Logistic Regression val accuracy:", logreg_acc)

plt.figure(figsize=(5, 4))
plt.bar(["Gaussian NB", "Logistic Regression"], [gnb_acc,
   logreg_acc], color=["#4C72B0", "#DD8452"])
plt.ylabel("validation accuracy")
plt.ylim(0, 1.05)
plt.title("Generative vs Discriminative model")
plt.savefig("../outputs/gen_vs_disc.png")
plt.close()

# ---------- Assignment 3: decision tree capacity ----------

depths = [1, 3, 5, None]
depth_labels = ["1", "3", "5", "None"]
train_accs = []
val_accs = []

for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_train)
    tr_acc = accuracy_score(y_train, tree.predict(X_train))
```

```
73      va_acc = accuracy_score(y_val, tree.predict(X_val))
74      train_accs.append(tr_acc)
75      val_accs.append(va_acc)
76      print("depth =", d, " train acc =", round(tr_acc, 3), " val
            acc =", round(va_acc, 3))
77
78  plt.figure(figsize=(6, 4))
79  plt.plot(depth_labels, train_accs, marker="o", label="train
        accuracy")
80  plt.plot(depth_labels, val_accs, marker="s", label="val accuracy
        ")
81  plt.xlabel("max_depth")
82  plt.ylabel("accuracy")
83  plt.ylim(0.5, 1.05)
84  plt.title("Decision tree accuracy vs depth")
85  plt.legend()
86  plt.savefig("../outputs/tree_capacity.png")
87  plt.close()
88
89  # ---------- Assignment 4: pick best depth using validation set
        ----------
90
91  best_i = val_accs.index(max(val_accs))
92  best_depth = depths[best_i]
93  print("best depth based on validation set:", best_depth)
94
95  final_tree = DecisionTreeClassifier(max_depth=best_depth,
        random_state=42)
96  final_tree.fit(X_train, y_train)
97  test_acc = accuracy_score(y_test, final_tree.predict(X_test))
98  print("final test accuracy:", test_acc)
99
100 # ---------- Assignment 5: bias/variance, just comparing shallow
        vs deep tree ----------
101
102 print("shallow tree (depth=1)  train:", train_accs[0], " val:",
        val_accs[0])
103 print("deep tree (depth=None)  train:", train_accs[3], " val:",
        val_accs[3])
104
105 # ---------- Assignment 6: cross entropy / log loss for logistic
        regression ----------
106
107 probs = logreg.predict_proba(X_val)
108 val_logloss = log_loss(y_val, probs)
109 print("validation log loss:", val_logloss)
110
111 preds = logreg.predict(X_val)
112 losses = []
113 for i in range(len(y_val)):
114     p = probs[i][y_val[i]]
```

```
115     losses.append(-np.log(p))
116
117 # find the most confident correct prediction and the worst wrong
        one
118 best_correct = None
119 worst_wrong = None
120 for i in range(len(y_val)):
121     if preds[i] == y_val[i]:
122         if best_correct is None or losses[i] < losses[
                best_correct]:
123             best_correct = i
124     else:
125         if worst_wrong is None or losses[i] > losses[worst_wrong
                ]:
126             worst_wrong = i
127
128 print("most confident correct sample:", best_correct,
        class_names[y_val[best_correct]], "loss =", losses[
        best_correct])
129 print("most confident wrong sample:", worst_wrong, "true:",
        class_names[y_val[worst_wrong]],
130       "predicted:", class_names[preds[worst_wrong]], "loss =",
            losses[worst_wrong])
131
132 order = sorted(range(len(losses)), key=lambda i: losses[i])
133 colors = ["#55A868" if preds[i] == y_val[i] else "#C44E52" for i
        in order]
134
135 plt.figure(figsize=(6.5, 4))
136 plt.bar(range(len(losses)), [losses[i] for i in order], color=
        colors)
137 plt.axhline(val_logloss, linestyle="--", color="black", label=f"
        mean loss = {val_logloss:.3f}")
138 plt.xlabel("validation samples (sorted by loss)")
139 plt.ylabel("cross entropy loss")
140 plt.title("Per-sample cross entropy (green = correct, red =
        wrong)")
141 plt.legend()
142 plt.savefig("../outputs/cross_entropy.png")
143 plt.close()
144
145 # ---------- Assignment 7: MLE vs MAP, effect of regularization
        strength C ----------
146
147 C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
148 weight_norms = []
149 val_accs_C = []
150
151 for C in C_values:
152     m = LogisticRegression(C=C, max_iter=2000, random_state=42)
153     m.fit(X_train, y_train)
```

```
    weight_norms.append(np.linalg.norm(m.coef_))
    val_accs_C.append(accuracy_score(y_val, m.predict(X_val)))
    print("C =", C, " weight norm =", round(weight_norms[-1], 3)
        , " val acc =", round(val_accs_C[-1], 3))

fig, ax1 = plt.subplots(figsize=(6.5, 4))
ax1.plot(C_values, weight_norms, marker="o", color="#4C72B0")
ax1.set_xscale("log")
ax1.set_xlabel("C (log scale)")
ax1.set_ylabel("weight norm ||w||", color="#4C72B0")

ax2 = ax1.twinx()
ax2.plot(C_values, val_accs_C, marker="s", color="#C44E52")
ax2.set_ylabel("validation accuracy", color="#C44E52")
ax2.set_ylim(0.5, 1.05)

plt.title("Effect of regularization strength on weights /
    accuracy")
fig.tight_layout()
fig.savefig("../outputs/mle_map.png")
plt.close()

# ---------- save everything needed for the report ----------

results = {
    "n_train": len(X_train),
    "n_val": len(X_val),
    "n_test": len(X_test),
    "gnb_val_acc": gnb_acc,
    "logreg_val_acc": logreg_acc,
    "tree_depths": depth_labels,
    "tree_train_acc": train_accs,
    "tree_val_acc": val_accs,
    "best_depth": str(best_depth),
    "final_test_acc": test_acc,
    "val_log_loss": val_logloss,
    "C_values": C_values,
    "weight_norms": weight_norms,
    "val_acc_vs_C": val_accs_C,
}

with open("../outputs/results.json", "w") as f:
    json.dump(results, f, indent=2)

print("done, results saved to outputs/results.json")
```